

Skill 8

Task 1

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
GNU nano 7.2 combine
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>
int main() {
    char command[200];
    char filename[100];
    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);
    // Remove newline
    command[strcspn(command, "\n")] = '\0';
    printf("Enter output file: ");
    scanf("%s", filename);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        // Open output file
        int fd = open(filename,
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }
    }
}
```

```

if (dup2(fd, STDOUT_FILENO) < 0) {
    perror("dup2 stdout");
    close(fd);
    exit(1);
}
/*
 * Then redirect stderr to the same
 * file descriptor as stdout.
 */
if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0) {
    perror("dup2 stderr");
    close(fd);
    exit(1);
}
close(fd);
// Execute command using shell
execl("/bin/sh", "sh", "-c", command, (char *)NULL);
// If execl fails
perror("execl");
exit(1);
}
else {
    // Parent waits for child
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        printf("Command finished with exit code: %d\n",
            WEXITSTATUS(status));
    }
}

```

Output:

```

root@Tejasri:~# nano combined.c
root@Tejasri:~# gcc combined.c -o combined
root@Tejasri:~# ./combined
Enter command: ls /home test
Enter output file: output.txt
Command finished with exit code: 2
root@Tejasri:~# |

```

Task 2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#define MAX 100
int main() {
    char command[500];
    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);
    // Remove newline
    command[strcspn(command, "\n")] = '\0';
    // Check for pipe
    char *pipe_pos = strchr(command, '|');
    if (pipe_pos == NULL) {
        printf("No pipe found.\n");
        return 0;
    }
    // Split command into two parts
    *pipe_pos = '\0';
    char *cmd1 = command;
    char *cmd2 = pipe_pos + 1;
    // Remove leading spaces
    while (*cmd2 == ' ')
        cmd2++;
    // Check for output redirection
    char *redirect = strchr(cmd2, '>');
    char output_file[100] = "";

```

```

// Check for output redirection
char *redirect = strchr(cmd2, '>');
char output_file[100] = "";
if (redirect != NULL) {
    *redirect = '\0';
    redirect++;
    while (*redirect == ' ')
        redirect++;
    strcpy(output_file, redirect);
}
// Parse first command
char *args1[MAX];
int count1 = 0;
char *token = strtok(cmd1, " ");
while (token != NULL) {
    args1[count1++] = token;
    token = strtok(NULL, " ");
}
args1[count1] = NULL;
// Parse second command
char *args2[MAX];
int count2 = 0;
token = strtok(cmd2, " ");
while (token != NULL) {
    args2[count2++] = token;
    token = strtok(NULL, " ");
}
args2[count2] = NULL;

```

```

}
// First child
pid_t pid1 = fork();
if (pid1 == 0) {
    // stdout -> pipe
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]);
    close(fd[1]);
    execvp(args1[0], args1);
    perror("execvp");
    exit(1);
}
// Second child
pid_t pid2 = fork();
if (pid2 == 0) {
    // stdin <- pipe
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    close(fd[1]);
    // Handle output redirection
    if (strlen(output_file) > 0) {
        int file = open(output_file,
                        O_WRONLY | O_CREAT | O_TRUNC,
                        0644);
        if (file < 0) {
            perror("open");
            exit(1);
        }
        // stdout -> file

```

```

        dup2(file, STDOUT_FILENO);
        close(file);
    }
    execvp(args2[0], args2);
    perror("execvp");
    exit(1);
}
// Parent closes pipe
close(fd[0]);
close(fd[1]);
// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
printf("Command executed successfully.\n");
return 0;
}

```

Output:

```

root@Tejasri:~# nano complex.c
root@Tejasri:~# gcc complex.c -o complex
root@Tejasri:~# ./complex
Enter command: cat input.txt | grep hello > result.txt
cat: input.txt: No such file or directory
Command executed successfully.
root@Tejasri:~# ./complex
Enter command: ls | grep .c > output.txt
Command executed successfully.
root@Tejasri:~# |

```

Task 3

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX 100
int main() {
    char command[500];
    printf("myshell> ");
    while (fgets(command, sizeof(command), stdin) != NULL) {
        // Remove newline
        command[strcspn(command, "\n")] = '\0';
        if (strlen(command) == 0) {
            printf("myshell> ");
            continue;
        }
        // Check whether command is a background job
        int background = 0;
        int len = strlen(command);
        if (command[len - 1] == '&') {
            background = 1;
            // Remove '&'
            command[len - 1] = '\0';
            // Remove trailing spaces
            len = strlen(command);
            while (len > 0 && command[len - 1] == ' ') {
                command[len - 1] = '\0';
                len--;
            }
        }
    }
}
```

```

}
// Exit shell
if (strcmp(command, "exit") == 0) {
    break;
}
// Store command arguments
char *args[MAX];
int count = 0;
char *token = strtok(command, " ");
while (token != NULL && count < MAX - 1) {
    args[count++] = token;
    token = strtok(NULL, " ");
}
args[count] = NULL;
if (args[0] == NULL) {
    printf("myshell> ");
    continue;
}
// Create child process
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
}
else if (pid == 0) {
    // Child process
    execvp(args[0], args);
    perror("execvp");
    exit(1);
}
}

```

```

pid_t pid = fork();
if (pid < 0) {
    perror("fork");
}
else if (pid == 0) {
    // Child process
    execvp(args[0], args);
    perror("execvp");
    exit(1);
}
else {
    // Parent process
    if (background) {
        printf("[Background Job] PID = %d\n", pid);
        /*
         * Do NOT wait here.
         * Parent immediately returns to prompt.
         */
    }
    else {
        // Foreground process
        waitpid(pid, NULL, 0);
    }
}
printf("myshell> ");
}
return 0;
}

```

Output:

```
root@Tejasri:~# nano background.c
root@Tejasri:~# gcc background.c -o background
root@Tejasri:~# ./background
myshell> ls
Commands      S5Task2.c      assets          error.txt       output_redirect  sTask.c         signals.c
Commands.c    Session3       background      export          output_redirect.c sTask1.c        skill
Dashboard     Session3.c    bin            export.c        output_redirection sTask2.c        skill.c
Dashboard.c   Shell         client         filecopy        output_redirection.c sTask3.c        source.txt
File1.tx      Task.save     client.c       filename.c      pipe             sTask4.c        state.txt
LICENSE       Task1         combined       gitignore       pipe.c           sTask5.c        src
LICENSE       Task1.c       combined.c     include         posix            sTask6.c        state.txt
Makefile      Task1.c.save  command_pipe.c incode.c        posix.c          sTask7.c        stderr_redirect
Memory        Task2         complex        input_redirection sTask8.c        stderr_redirect.c
Memory.c      Task2.c       complex.c      input_redirection.c sTask9.c        stderr_redirect.c
MyShell       Task2.c.save  copy          links.c          program.c        scripts
MyShell.c     Task3         copy.c         map              program.c        server
OSSP          Task3.c       cow.c          map.c            race_condition   server.c         tests
Ptask1        Task4.c       cow.c          mutex_counter    race_condition.c session3.c        week1.c.save
Ptask1.c      Week3.c.save  deadlock.c     mutex_counter.c redirect          session3.c.save week2
Ptask2        append_redirect  destination.txt navigation        redirect.c       shell.c          week2.c
Ptask2.c      append_redirect.c  directory      obj              redirection      shell.c          week5.save
README.md     append_redirection  directory.c    output.txt       redirection.c    shopping
S5Task1       append_redirection.c  directory.c    output.txt       redirection.c    shopping.c
S5Task1.c     append_redirection.c  docs           output.txt       result.txt       shopping.c.save
S5Task2       assets        docs           output.txt       sTask            signals
myshell> sleep 10
myshell> sleep 10 &
[Background Job] PID = 4566
myshell> |
```

Task 4

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
GNU nano 2.9.2 jobtable.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_JOBS 100
// Job structure
struct Job {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char status[20];
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
// Add a new job
void add_job(pid_t pid, pid_t pgid, char *command) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");
    job_count++;
}
```

```

}
// Display job table
void display_jobs() {
    printf("\n%-5s %-8s %-8s %-20s %-12s\n",
           "Job", "PID", "PGID", "Command", "Status");
    printf("-----\n");
    for (int i = 0; i < job_count; i++) {
        printf("%-5d %-8d %-8d %-20s %-12s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].pgid,
               jobs[i].command,
               jobs[i].status);
    }
    printf("\n");
}
// Update job status
void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
        if (result > 0) {
            if (WIFEXITED(status)) {
                strcpy(jobs[i].status, "Completed");
            }
        }
    }
}

```

```

}
// Display job table
void display_jobs() {
    printf("\n%-5s %-8s %-8s %-20s %-12s\n",
           "Job", "PID", "PGID", "Command", "Status");
    printf("-----\n");
    for (int i = 0; i < job_count; i++) {
        printf("%-5d %-8d %-8d %-20s %-12s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].pgid,
               jobs[i].command,
               jobs[i].status);
    }
    printf("\n");
}
// Update job status
void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
        if (result > 0) {
            if (WIFEXITED(status)) {
                strcpy(jobs[i].status, "Completed");
            }
        }
    }
}

```

```

/ Remove completed jobs
oid remove_completed_jobs() {
    for (int i = 0; i < job_count; i++) {
        if (strcmp(jobs[i].status, "Completed") == 0 ||
            strcmp(jobs[i].status, "Terminated") == 0) {
            // Shift remaining jobs
            for (int j = i; j < job_count - 1; j++) {
                jobs[j] = jobs[j + 1];
            }
            job_count--;
            i--;
        }
    }
}

/ Validate job records
oid validate_jobs() {
    printf("\nValidating job records...\n");
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].pid > 0 &&
            jobs[i].pgid > 0 &&
            strlen(jobs[i].command) > 0) {
            printf("Job %d: Valid\n",
                jobs[i].job_id);
        }
        else {
            printf("Job %d: Invalid\n",
                jobs[i].job_id);
        }
    }
}

```

```

GNU nano 7.2 jobcat.c
int main() {
    int choice;
    char command[100];
    while (1) {
        printf("\n===== JOB TABLE =====\n");
        printf("1. Start Job\n");
        printf("2. Display Jobs\n");
        printf("3. Update Status\n");
        printf("4. Remove Completed Jobs\n");
        printf("5. Validate Jobs\n");
        printf("6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        if (choice == 1) {
            printf("Enter command: ");
            fgets(command, sizeof(command), stdin);
            command[strcspn(command, "\n")] = '\0';
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork");
                continue;
            }
            if (pid == 0) {
                // Create process group
                setpgid(0, 0);
                execl("/bin/sh",
                    "sh",
                    "-c",

```

```

    }
    else {
        // Parent creates process group
        setpgid(pid, pid);
        add_job(pid, pid, command);
        printf("Job added successfully.\n");
        printf("PID: %d\n", pid);
        printf("PGID: %d\n", pid);
    }
}
else if (choice == 2) {
    update_jobs();
    display_jobs();
}
else if (choice == 3) {
    update_jobs();
    printf("Job status updated.\n");
    display_jobs();
}
else if (choice == 4) {
    update_jobs();
    remove_completed_jobs();
    printf("Completed jobs removed.\n");
    display_jobs();
}
else if (choice == 5) {
    validate_jobs();
}
else if (choice == 6) {

```

Output:

```

root@Tejasri:~# nano jobtable.c
root@Tejasri:~# gcc jobtable.c -o jobtable
root@Tejasri:~# ./jobtable

```

```

===== JOB TABLE =====
1. Start Job
2. Display Jobs
3. Update Status
4. Remove Completed Jobs
5. Validate Jobs
6. Exit
Enter choice: 2

```

Job	PID	PGID	Command	Status

```

===== JOB TABLE =====
1. Start Job
2. Display Jobs
3. Update Status
4. Remove Completed Jobs
5. Validate Jobs
6. Exit
Enter choice: 5

```

```

Enter choice: 5
Validating job records...

===== JOB TABLE =====
1. Start Job
2. Display Jobs
3. Update Status
4. Remove Completed Jobs
5. Validate Jobs
6. Exit
Enter choice: 6
Exiting...
root@Tejasri:~#

```